Assignment 1

ME45155 Computational Fluid Dynamics for Engineers

2026-12-03

Instructions

- **Submit on Brightspace** under Assingments/Assignment 1 (missing one of the two items below results in an invalid submission):
    1. **The filled in answer sheets provided at the end of this document**. For Figures, you may choose to digitally paste them onto the answer sheet, or to upload them in a separate pdf file. In the latter case, please make sure to reference the Question number associated with the figure. **You do not have to submit the assignment description**.
    2. **The codes you have written to solve the problems** (Python/Jupyter notebook files, or any other programming language you prefer).
- **Please aim for concise, content-rich answers**. We will assess your understanding, reasoning, and technical knowledge rather than the neatness or stylistic polish of the write-up, so brief telegraphic answers are perfectly acceptable as long as they are clear and address the question!
- **Submission deadline: 9th April 2026, 17:30.**

Problem 1 – Solving a maze using the diffusion equation

Consider a water-filled maze whose entrance is kept in contact with a dye source, while the exit is kept in contact with clean water. With the resulting concentration difference, dye spreads through the maze by diffusion. Over time, the concentration field shows how easily different parts of the maze can be reached. You may also see the problem as a simplified model for the fascinating phenomenon shown in this video of a slime mould solving a maze.

To simulate this process, we will solve the variable-coefficient unsteady diffusion equation in a two-dimensional Cartesian domain:

$$\frac{\partial c}{\partial t} = \nabla \cdot (\alpha \nabla c) = \frac{\partial}{\partial x}\left(\alpha \frac{\partial c}{\partial x}\right) + \frac{\partial}{\partial y}\left(\alpha \frac{\partial c}{\partial y}\right),$$

where $c(x, y, t)$ is the concentration of the dye, and $\alpha(x, y)$ is the diffusion coefficient. The maze geometry has a domain $L_x \times L_y = 1 \times 1$, and is described by a binary field

$$I(x, y) = \begin{cases} 0, & \text{for } (x, y) \text{ in liquid-filled corridors,} \\ 1, & \text{for } (x, y) \text{ in maze walls.} \end{cases}$$

This field is provided in the file `maze_geometry.npy` with a resolution $N_x \times N_y = 50 \times 50$ on a *cell-centered* grid defined by:

$$x_i = (i - 1/2)\,\Delta x, \quad i = 1, 2, \ldots, N_x,$$
$$y_j = (j - 1/2)\,\Delta y, \quad j = 1, 2, \ldots, N_y.$$

with $\Delta x = L_x/N_x$ and $\Delta y = L_y/N_y$. This file can be loaded and visualized as follows:

```
import numpy as np
import matplotlib.pyplot as plt

Lx = Ly = 1.0
Nx = Ny = 50
dx = Lx / Nx
dy = Ly / Ny
x = np.linspace(0.5 * dx, Lx - 0.5 * dx, Nx)
y = np.linspace(0.5 * dy, Ly - 0.5 * dy, Ny)
X, Y = np.meshgrid(x, y)  # Cell-center coordinates associated with I(x, y)

I = np.load("maze_geometry.npy")

im = plt.imshow(
    I,
    origin="lower",
    extent=[0.0, Lx, 0.0, Ly],
    cmap="binary",
    vmin=0,
    vmax=1,
    interpolation="nearest",
    aspect="equal",
)
```

```
plt.colorbar(im, ticks=[0, 1], label="$I(x,y)$")
plt.xlabel("x")
plt.ylabel("y")
plt.tight_layout()
plt.show()
```

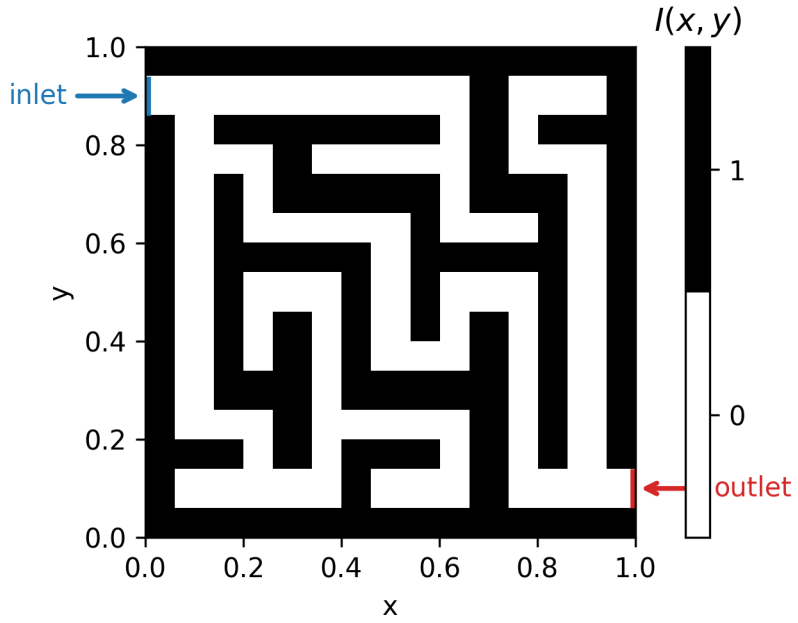

Figure 1: Visualization of the maze geometry. Black cells are walls ($I(x, y) = 1$), while white cells are liquid-filled corridors ($I(x, y) = 0$). The inlet is the highlighted opening on the left boundary, and the outlet is the highlighted opening on the right boundary.

> **Note**
>
> Though it is not necessary for plotting the maze geometry with `imshow`, the code for defining the coordinate arrays `X[j, i]` and `Y[j, i]` is included above, as it will be useful for the numerical implementation. In NumPy indexing, the first index corresponds to `y` and the second to `x`. Ghost cells are not included in this grid definition, so you may need to slightly adjust it.

To model the effect of the solid walls, consider a much lower diffusion coefficient in the walls than in the liquid-filled corridors[1]. Assume that the diffusion coefficient in the walls is $\alpha_{\text{wall}} = 10^{-8}$ and that in the liquid it is $\alpha_{\text{liquid}} = 10^{-1}$. This can be expressed in terms of our $I(x, y)$ field as:

$$\alpha(x, y) = \alpha_{\text{liquid}}(1 - I(x, y)) + \alpha_{\text{wall}} I(x, y)$$

We consider a constant concentration of dye at the inlet, $c_{\text{in}} = 1$, and at the outlet, $c_{\text{out}} = 0$. From the maze geometry shown in the figure above, the inlet is the opening on the left boundary and the outlet is

[1] You may even set it to zero in the present assignment, but if we were to solve the steady diffusion equation, we would get a singular matrix with $\alpha_{\text{wall}} = 0$.

the opening on the right boundary. Their locations are approximately given by:

$$\begin{cases} x_{\text{in}} = 0, & 0.86 < y_{\text{in}} < 0.94, \\ x_{\text{out}} = 1, & 0.06 < y_{\text{out}} < 0.14. \end{cases}$$

We want the dye to spread through a maze that is initially filled with only clean water, so we must consider a zero concentration field as the initial condition:

$$c(x, y, 0) = 0.$$

Question 1.1 [4/100 pts.].

- Why does considering a much lower diffusion coefficient in the walls, combined with these inlet and outlet boundary conditions, lead to the dye spreading predominantly through the maze corridors?
- Which kind of boundary conditions should one apply at the other domain boundaries – $x = 0$, $x = 1$, $y = 0$, $y = 1$ that are not the inlet and outlet – to ensure that there is no mass flux across them? Draw a sketch of the contour of the domain boundary[2] and annotate the boundary conditions in their continuum form (not yet discretized).

Question 1.2 [7/100 pts.].

Since values of $\alpha(x, y)$ are discontinuous across walls inside the domain, we need to be cautious when discretizing the diffusion equation. Consider two neighboring cells in the 1D grid below, at i and $i + 1$, one being liquid ($\alpha = \alpha_{\text{l}}$) and the other being a wall ($\alpha = \alpha_{\text{w}}$). Discretize the 1D diffusive flux $\alpha \nabla c$ using:

- A one-sided difference approximation within the liquid-only region, i.e., between cell center `i` and the cell face `i+1/2`.
- A one-sided difference approximation within the wall-only region, i.e., between the cell face `i+1/2` and cell center `i+1`.
- A central-difference approximation across the wall-liquid interface, considering that the diffusion coefficient at the interface is approximately α^*.

Determine the value of α^* in terms of α_{l} and α_{w} that ensures a continuous flux across the wall-liquid interface for the last bullet above.

> **Note**
>
> α^* will give you the type of interpolation needed to achieve a continuous flux across wall-liquid interfaces! You can also verify that a simple arithmetic average does not lead to a flux continuity.

[2] Excluding the internal maze walls, which numerically are treated as a lower-diffusion medium, not boundaries!

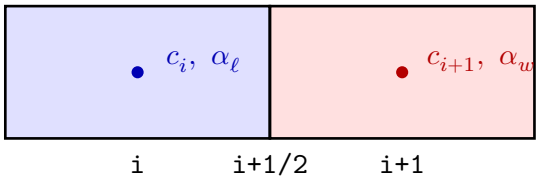

Figure 2: Sketch of two neighboring cells separated by the interface at $i + 1/2$.

Question 1.3 [7/100 pts.].

Sketch the computational grid for a number of grid points (unknowns) along x and y given by $N_x \times N_y = 4 \times 3$, considering a cell-centered grid with ghost cells for boundary conditions, and indicate the location of the boundaries, the cell centers, and the cell faces.

Then derive the spatial discretization of the diffusion term (**2D**) using a finite difference or a finite-volume approach (your choice) for an internal cell i, j. State which approach you chose when deriving it, and annotate your indexing notation on your sketched grids.

Question 1.4 [7/100 pts.].

Derive the boundary conditions for ghost cells in the discretization in Question 1.3 for the inlet, the outlet, and the remaining boundaries. Note that we have indicated above the location of the inlet and outlet boundaries.

The corresponding implementation in the code could look like:

```
ymin_in, ymax_in = 0.86, 0.94
ymin_out, ymax_out = 0.06, 0.14

for j in range(1, Ny + 1):
    yj = (j - 0.5) * dy

    if yj >= ymin_in and yj <= ymax_in:
        c[j, 0] = ...
    else:
        c[j, 0] = ...

    if yj >= ymin_out and yj <= ymax_out:
        c[j, Nx + 1] = ...
    else:
        c[j, Nx + 1] = ...

c[0, :] = ...
c[Ny + 1, :] = ...
```

Check if the code logic is consistent with your answer. You can then complete it and use it as base for boundary condition imposition in your code (no need to report the code here, as you will upload your final code in the assignment submission).

At this stage, we are ready to implement the discretization in code, but first let's think about the time step size.

Question 1.5 [7/100 pts.].

Choose a temporal integration method to solve the diffusion equation, combined with the initial condition, spatial discretization, and boundary conditions derived in previous questions[3]

- Assume that you or a colleague chose a Forward Euler integrator. What is the stability restriction for the time step size Δt for this integrator in terms of Δx, Δy, and the diffusion coefficient? Note that your expression should make clear which value of α controls the most restrictive explicit time-step limit in this problem. With this time-step stability limit, how many time steps would you need to reach $t = 1$, 5, 15, and 50?
- How would your answer change for an implicit integrator such as Backward Euler or Crank-Nicolson? In that case, how would you estimate a time step size that would ensure an accurate time integration? (*hint*: what is the most relevant physical time scale for this problem?)

Question 1.6 [11/100 pts.].

Starting from the code snippets provided here, implement your discretization of the diffusion equation together with the ghost-cell boundary conditions derived above. Feel free to re-use code from previous lectures as relevant.

Keep the spatial resolution that matches the way the maze has been digitized: $N_x \times N_y = 50 \times 50$, so that you do not have to use interpolation of the maze geometry to a new grid.

- Plot the concentration field $c(x, y, t)$ at $t \approx 1.0, 5.0, 15.0$ and, optionally[4], 50 and share the results.
- Briefly describe the evolution of the concentration field over time. In particular, comment on how the dye is transported through the maze.

Recall that you should use the interpolation derived in Question 1.2 to discretize the diffusive fluxes.

You may find it useful to visualize the transient solution by plotting a few temporal snapshots, or by making a simple animation during the time loop as shown below (tweak the `skip` / `ipause` parameters to control the animation frame rate):

```
plt.figure()

for n in range(nt):
    # update c here

    skip = 200
    ipause = 0.01
    if n % skip == 0:
```

[3] Your choice may be as simple as a Forward Euler method, but you are free to choose a more advanced explicit or implicit method.

[4] Depending on the time integration scheme you choose, that may be achieved very fast, or take significantly long.

```
        plt.clf()
        maze_walls = np.ma.masked_where(I == 0, I)
        plt.imshow(
            c,
            origin="lower",
            extent=[0.0, Lx, 0.0, Ly],
            cmap="viridis",
            aspect="equal",
        )
        plt.imshow(
            maze_walls,
            origin="lower",
            extent=[0.0, Lx, 0.0, Ly],
            cmap="gray_r",
            alpha=0.15,
            interpolation="nearest",
            aspect="equal",
        )
        plt.colorbar()
        plt.xlabel("x")
        plt.ylabel("y")
        plt.title("Transient concentration")
        plt.pause(ipause)
plt.show()
```

Question 1.7 [7/100 pts.].

- Identify a quantity for the maze problem that you expect to be approximately constant along the main path from inlet to outlet once the solution is close to steady state, and compute it at the cell centers using second-order central finite differences. Explain your choice.

> **Hint**
>
> Remember the one-dimensional conduction problem through a bar with two prescribed temperatures at the ends. In the steady state limit, which important quantity is constant along the bar?

- Why does this quantity gradually reveal the preferred path from inlet to outlet, and why does it not persist in dead ends at longer times?
- Report the steady-state (or nearly steady-state) plot of this quantity, and if you have done it right, that should yield the maze's solution.

> **Note**
>
> For a cleaner visualization, compute this quantity only at cell centers whose full central-difference stencil remains inside the fluid region. In other words, if the current cell or any of its immediate neighbors needed for the stencil lies inside a wall, set the diagnostic quantity to zero instead of evaluating it there:

```
diagnostic_quantity = np.zeros((Ny, Nx))

for j in range(1, Ny - 1):
    for i in range(1, Nx - 1):
        if (
            I[j, i] == 0
            and I[j, i + 1] == 0
            and I[j, i - 1] == 0
            and I[j + 1, i] == 0
            and I[j - 1, i] == 0
        ):
            # compute diagnostic quantity here
        else:
            diagnostic_quantity[j, i] = 0.0 # redundant, as already set to 0. before the
 ↪  loop
```

You may also find it helpful to overlay the maze walls lightly on top of this diagnostic plot:

```
maze_walls = np.ma.masked_where(I == 0, I)

plt.imshow(diagnostic_quantity, origin="lower", extent=[0.0, Lx, 0.0, Ly], cmap="magma",
 ↪  aspect="equal")
plt.imshow(maze_walls, origin="lower", extent=[0.0, Lx, 0.0, Ly], cmap="gray_r",
 ↪  alpha=0.15, interpolation="nearest", aspect="equal")
plt.colorbar()
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Problem 2 – Forced convection in a differentially heated cavity

We now consider a second model problem inspired by a differentially heated room with a fan mounted near the left side. The left wall is hot, the right wall is cold, and the fan drives a large-scale recirculating motion that dominates over any natural-convection effects due to gravity. In the present exercise, we do *not* solve the full Navier-Stokes equations. Instead, we prescribe a smooth incompressible velocity field, of Taylor-Green-vortex type, that mimics this forced circulation, and we solve the advection-diffusion equation for the temperature field.

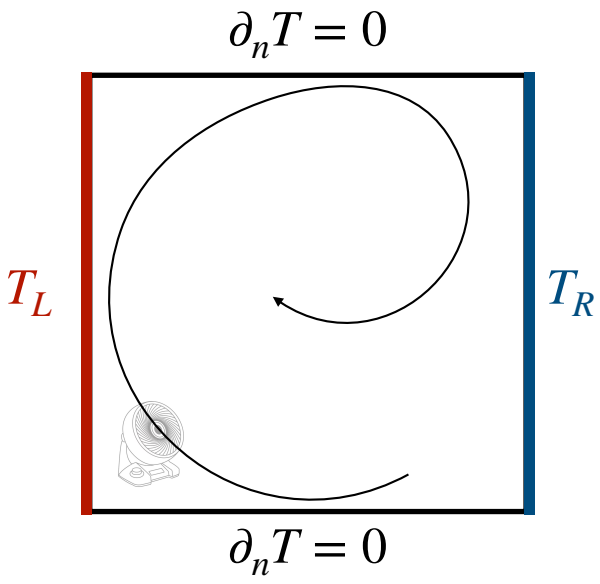

Figure 3: Schematic of the forced-convection cavity considered in Problem 2.

We consider a square cavity with domain $L_x \times L_y = 1 \times 1$, temperature field $T(x, y, t)$, and constant thermal diffusivity α. The governing equation is

$$\frac{\partial T}{\partial t} + \frac{\partial (uT)}{\partial x} + \frac{\partial (vT)}{\partial y} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right),$$

where the prescribed incompressible velocity field is

$$u(x, y) = -U_0 \sin(\pi x) \cos(\pi y), \qquad v(x, y) = U_0 \cos(\pi x) \sin(\pi y).$$

The left wall is maintained at a hot temperature T_{L}, the right wall is maintained at a cold temperature T_{R}, and the top and bottom walls are adiabatic:

$$T(0, y, t) = T_{\mathrm{L}}, \qquad T(L_x, y, t) = T_{\mathrm{R}}, \qquad \frac{\partial T}{\partial y}(x, 0, t) = \frac{\partial T}{\partial y}(x, L_y, t) = 0.$$

The fan is only used to motivate the imposed recirculating velocity field above; it does not introduce an additional thermal boundary condition. More realistic thermal boundary conditions could be imposed, e.g., through Newton's law of cooling, leading to Robin boundary conditions. However, for this assignment we stick to simpler boundary conditions.

Assume the cavity is initially cold:

$$T(x, y, 0) = T_{\mathrm{R}}.$$

Since α is now constant, you may recycle the code from Problem 1 and simplify the variable-coefficient diffusion operator accordingly (or re-use code from previous assignments/exercises/lectures). For the first implementation and debugging stage, I suggest to start with a coarse grid of 20×20 points, and then move to finer grids as requested in Question 2.4.

The following snippet illustrates the basic setup along with the considered physical parameters:

```
import numpy as np

Lx = Ly = 1.0
Nx = Ny = 20
dx = Lx / Nx
dy = Ly / Ny

alpha = 1.0e-3
U0 = 1.0
T_left = 1.0
T_right = 0.0

x = np.linspace(0.5 * dx, Lx - 0.5 * dx, Nx)
y = np.linspace(0.5 * dy, Ly - 0.5 * dy, Ny)
X, Y = np.meshgrid(x, y)

u = -U0 * np.sin(np.pi * X) * np.cos(np.pi * Y)
v = U0 * np.cos(np.pi * X) * np.sin(np.pi * Y)

T = np.full((Ny + 2, Nx + 2), T_right)
```

Question 2.1 [14/100 pts.].

- Select a spatial discretization method: finite-difference or finite-volume. If you choose finite differences, you may also choose whether to expand the transport term using the product rule and work in non-conservative form (note that the present velocity field is divergence-free).
- State the scheme used to discretize the transport terms (e.g., upwind-based, central scheme, …).
- Derive the fully discretized equation for a grid cell (i, j), together with the update from time level n to $n + 1$. For diffusion, you may opt to refer to your answer to Question 1.3, as it should also be valid for constant alpha.

> Tip
>
> Since the velocity field is prescribed analytically, you may simply evaluate the velocity components directly at the cell faces instead of interpolating them from neighboring cell centers.

Question 2.2 [14/100 pts.].

As in the previous assignment, assume you integrate the equation in time using Forward Euler.

- Define an advective Courant, Fourier, and cell Peclet number(s) for this problem.
- Explain how these quantities constrain the time step if you want the solution to remain stable and bounded *for the specific spatial discretization scheme you have chosen.*

Question 2.3 [14/100 pts.].

Implement the discretization in code to calculate the temperature evolution until steady state is reached. A way to visualize both temperature and the prescribed fan-driven circulation is to use a quiver plot, as shown below:

```
plt.figure()
plt.imshow(
    T[1:-1, 1:-1],
    origin="lower",
    extent=[0.0, Lx, 0.0, Ly],
    cmap="inferno",
    aspect="equal",
)
plt.quiver(X, Y, u, v, color="white")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

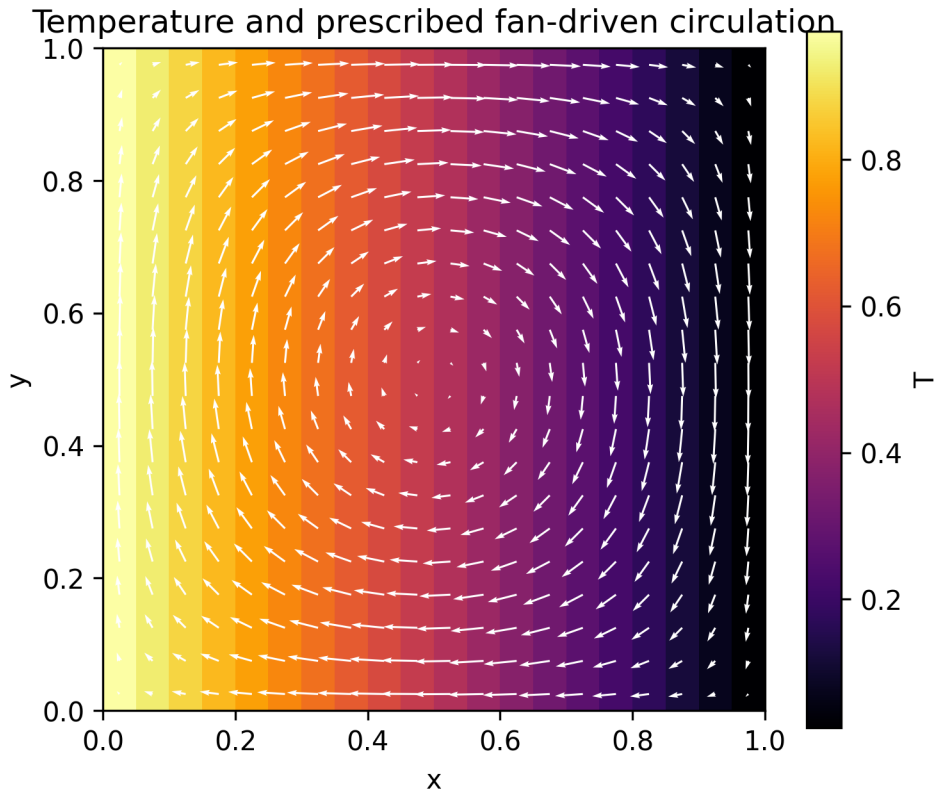

Figure 4: Example of the prescribed fan-driven circulating velocity field plotted on top of a linearly-varying temperature field.

Then:

- Present a plot of the steady-state temperature field, and report, approximately, the physical time it took to reach steady-state. How did you assess the solution's stationarity?
- Extract and plot the two centreline temperature profiles ($T(x, y = 0.5)$ vs x and $T(x = 0.5, y)$ vs y) and comment on their shape near the boundaries – are they consistent with the imposed boundary conditions?

Question 2.4 [8/100 pts.].

Perform a grid convergence study using the systematically refined grids

$$N_x = N_y = 20,\ 40,\ 60,\ 80.$$

To check for grid convergence, inspect the previously plotted temperature profiles along the centerlines, by re-creating those two plots with all four resolutions on the same figure.

- Which grid provides satisfactory practical grid convergence? Expand on the criterion you used to assess this.

Bonus exercise (not mandatory, and not evaluated)

Suppose the forced convection velocity must deliver a minimum wall-averaged heat-transfer performance, represented by a target Nusselt number $\overline{\mathrm{Nu}}_{\mathrm{target}}$. A convenient dimensionless definition is

$$\overline{\mathrm{Nu}}_L(t) = -\frac{1}{L_y\,(T_{\mathrm{L}} - T_{\mathrm{R}})} \int_0^{L_y} \alpha\, \frac{\partial T}{\partial x}\Big|_{x=0} \mathrm{d}y, \qquad \overline{\mathrm{Nu}}_R(t) = \frac{1}{L_y\,(T_{\mathrm{L}} - T_{\mathrm{R}})} \int_0^{L_y} \alpha\, \frac{\partial T}{\partial x}\Big|_{x=L_x} \mathrm{d}y.$$

Compute $\overline{\mathrm{Nu}}_L(t)$ and $\overline{\mathrm{Nu}}_R(t)$ for at least two values of U_0, and compare the time at which the Nusselt-number histories settle and agree with the steady-state criterion you used in Question 2.3, and comment on whether the two indicators are consistent.

Answer Sheets

The answer boxes for all questions are collected in the upcoming pages, and the *AI Use Declaration* page below.

AI Use Declaration

Please complete this declaration before submitting your answer sheets.

Student names: _____

Student numbers: _____

For this Assignment 1 submission, we declare that:

☐ We did not use generative AI tools in preparing this submission.

☐ We used generative AI tools for support tasks such as language polishing, formatting help, or debugging assistance, and we take full responsibility for the content of the submitted work.

Optional: if generative AI was used, state the purpose of that use (for example, language editing, debugging, etc.) and, if relevant, briefly how it was used:

We understand that all submitted work must reflect our own understanding of the methods, modelling choices, code, and results presented in this assignment. We also understand that, if there are concerns about academic misconduct in the submitted work, the course team reserves the right to ask the group to explain and justify their implementation, derivations, and results.

Question 1.1

Question 1.2

Question 1.3

Question 1.4

Question 1.5

Question 1.6

Question 1.7

Question 2.1

Question 2.2

Question 2.3

Question 2.4

Feedback

This section is optional, but your feedback would be really appreciated.

- Which parts of the assignment felt appropriately challenging, and which parts felt too easy or too difficult?
- Which parts of the assignment felt most relevant to your learning?
- Please share any suggestions that you believe would improve an assignment of this nature in the future.
- Any other comments you would like to share?

Additional Answer Space

Use this page only if one of your answers did not fit in the allocated answer sheets. Please clearly indicate the question number.

Additional Answer Space

Use this page only if one of your answers did not fit in the allocated answer sheets. Please clearly indicate the question number.